CSL301 : Operating System

LAB-8

Hands-on 1: Toll Plaza Tag Reader

Problem

A toll plaza uses a FASTag reader with a small local memory that can store K vehicle IDs. When a vehicle arrives, the reader first checks its local memory:

- If the vehicle ID is found in memory, the vehicle passes in **1 second**.
- If it is not found, the reader queries the central server, which takes **5 seconds**. The vehicle ID is then stored in local memory.

When the memory is full and a new vehicle ID must be stored, the reader removes the vehicle ID that was stored earliest. A vehicle that is found in memory does not change the order in which IDs were stored.

Task

Write a program that:

- computes the total time taken for the whole sequence of vehicles with K memory slots,
- prints the vehicle IDs present in memory at the end, in the order they were stored (earliest first),
- finds the minimum number of memory slots needed so that the total time is at most T seconds. If no number of slots achieves this, print "Not possible".

Input Format

- Line 1: K, the number of memory slots
- Line 2: N, the number of vehicles
- Line 3: N vehicle IDs (integers)
- Line 4: T, the time limit in seconds

Output Format

```
Total time with K slots: X seconds
Memory at end: id1 id2 ...
Minimum slots for time <= T: S
```

Sample

Sample Input	Sample Output
3 7 101 205 101 330 412 205 101 25	Total time with 3 slots: 27 seconds Memory at end: 330 412 101 Minimum slots for time <= 25: 4

Constraints

- $1 \leq K \leq 100$, $1 \leq N \leq 1000$

Think About

- Does adding more memory slots always reduce the total time? Can you find an input where it does not?

Hands-on 2: Phone RAM Manager

Problem

A mobile phone can keep K apps in RAM at a time. When the user opens an app:

- If the app is already in RAM, it is a **warm start** and takes **1 second**.
- If it is not in RAM, it is a **cold start** and takes **4 seconds**. The app is then loaded into RAM.

When RAM is full and a new app must be loaded, the phone kills the app that has not been opened for the longest time.

Task

Given K and the sequence of apps opened by the user, write a program that prints:

- the total time spent opening apps,
- the names of the apps killed, in the order they were killed (print "none" if no app was killed),
- the Recent Apps screen at the end: all apps in RAM, with the most recently opened app first.

Input Format

- Line 1: K, the number of apps RAM can hold
- Line 2: N, the number of times an app is opened
- Line 3: N app names (each a single word, at most 31 characters)

Output Format

```
Total time: X seconds
Apps killed: app1 app2 ...
Recent apps: app1 app2 ...
```

Sample

Sample Input	Sample Output
3 7 WhatsApp Chrome YouTube WhatsApp Maps Chrome WhatsApp	Total time: 22 seconds Apps killed: Chrome YouTube Recent apps: WhatsApp Chrome Maps

Constraints

- $1 \le K \le 100$, $1 \le N \le 100$. App names are case-sensitive.

Note

App names are strings, not numbers. Use a two-dimensional character array and the functions strcmp and strcpy from string.h.

Hands-on 3: Mechanic's Tool Belt

Problem

A mechanic carries a tool belt that can hold K tools. All other tools are kept in the van. At the start of the day, the mechanic knows the complete list of tools needed, in order, for all the day's jobs.

Whenever the next tool needed is not on the belt, the mechanic makes one trip to the van to take it. If the belt is already full, the mechanic must return one tool from the belt during the same trip.

The mechanic wants to make the fewest possible trips. If several tools on the belt will never be needed again, the mechanic returns the one that was put on the belt earliest. The belt is empty at the start of the day.

Task

Write a program that:

- prints every trip: which tool is taken, and which tool is returned (if any),
- prints the total number of trips,
- prints how many trips the mechanic would need if, instead, he always returned the tool that had not been used for the longest time.

Input Format

- Line 1: K, the number of tools the belt can hold
- Line 2: N, the number of tool uses in the day
- Line 3: N tool names, in the order they are needed (each a single word)

Output Format

```
Trip 1: take tool
Trip 2: take tool, return tool
...
Total trips: X
Trips if returning least recently used tool: Y
```

Sample

Sample Input	Sample Output
2 7 hammer wrench drill hammer wrench saw hammer	Trip 1: take hammer Trip 2: take wrench Trip 3: take drill, return wrench Trip 4: take wrench, return drill Trip 5: take saw, return wrench Total trips: 5 Trips if returning least recently used tool: 7

Constraints

- $1 \le K \le 100$, $1 \le N \le 100$

Think About

- Why can this strategy be used by the mechanic but not by an operating system?

Hands-on 4: Offline Music Storage

Problem

A music app lets users keep songs offline in a storage space of C MB. Unlike memory pages, songs have different sizes.

When the user plays a song:

- If the song is already stored, it is a hit.
- If it is not stored, it is a miss: the song is downloaded and stored. If there is not enough free space, stored songs are deleted one at a time, following the deletion policy, until the new song fits. Several songs may need to be deleted for one download.
- If a song is larger than C MB, it can never be stored. It is streamed instead: count it as a miss, but do not store it and do not add it to the downloaded total.

The app developers want to compare two deletion policies:

- **FIFO:** delete the song that was downloaded earliest.
- **LRU:** delete the song that has not been played for the longest time.

Task

Simulate both policies on the same sequence of plays (storage is empty at the start). For each policy, print the number of hits, the number of misses, the total MB downloaded, and the songs stored at the end. Then print the policy that downloaded less data, or "Both equal".

Print the stored songs in deletion order: the song that would be deleted next comes first.

Input Format

- Line 1: C, the storage capacity in MB
- Line 2: S, the number of songs in the library
- Next S lines: a song name (single word) and its size in MB
- Next line: P, the number of plays
- Next line: P song names, in the order they are played

Output Format

```
LRU:  hits H, misses M, downloaded D MB, stored: song1 song2 ...
FIFO: hits H, misses M, downloaded D MB, stored: song1 song2 ...
Better policy: LRU / FIFO / Both equal
```

Sample

Sample Input	Sample Output
10 4 A 4 B 3 C 2 D 5 7 A B C A D A B	LRU: hits 2, misses 5, downloaded 17 MB, stored: A B FIFO: hits 1, misses 6, downloaded 21 MB, stored: A B Better policy: LRU

Constraints: $1 \le C \le 10000$, $1 \le S \le 100$, $1 \le P \le 100$

Think about: Try an input where a large song forces two or more smaller songs to be deleted. Does your program delete exactly as many songs as needed, and no more?